



---

# PROJE TASARIM RAPORU

---

**Ders:** BIL204 / Yazılım Mühendisliği

**Bölüm/Şube:** Bilgisayar Mühendisliği

**Grup Adı:** UniSocial Team

**Proje Adı:** UniSocial - Üniversite Öğrencilerine Özel Sosyal Ağ Platformu

**Teslim Tarihi:** 04.05.2026

**Grup Üyeleri:**

- Mehmet Halil Muğulday (25100011412)
- Yağmur Nisa Süslü (24100011049)
- Büşra Taşpınar (24100011057)
- Emine Sude Alişarlı (23100011035)
- Sudem Arslan (24100011061)

## İçindekiler

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>1. Giriş</b>                                                        | <b>4</b>  |
| 1.1. Sistemin Amacı                                                    | 4         |
| 1.2. Tasarım Hedefleri                                                 | 4         |
| 1.2.1. Güvenlik(Security)                                              | 4         |
| 1.2.2. Kullanılabilirlik(Usability)                                    | 4         |
| 1.2.3. Bakım Yapılabilirlik ve Geliştirilebilirlik(Maintainability)    | 5         |
| <b>2. Üst Düzey Yazılım Mimarisi</b>                                   | <b>6</b>  |
| 2.1. Alt Sistem Ayrıştırması                                           | 6         |
| 2.2. Donanım/Yazılım Eşlemesi                                          | 7         |
| 2.3. Kalıcı Veri Yönetimi                                              | 8         |
| 2.3.1. Kalıcı Olarak Saklanacak Veriler                                | 9         |
| 2.3.2. Veri Tabanı Teknolojisi ve Seçim Gerekçeleri                    | 9         |
| 2.3.3. Veri Erişim Stratejisi                                          | 9         |
| 2.3.4. ER Diyagramı                                                    | 11        |
| 2.4. Erişim Kontrolü ve Güvenlik                                       | 11        |
| 2.4.1.Erişim Kontrol Matrisi                                           | 12        |
| 2.4.2. Kimlik Doğrulama (Authentication) Süreçleri                     | 14        |
| 2.4.3. Yetkilendirme (Authorization) ve Erişim Sınırları               | 14        |
| 2.4.4. Veri Güvenliği ve Alınan Ek Önlemler                            | 15        |
| 2.5. Küresel Kontrol Akışı                                             | 15        |
| 2.6. Sınır Koşulları                                                   | 16        |
| 2.6.1. Başlatma                                                        | 16        |
| 2.6.2. Sonlandırma                                                     | 16        |
| 2.6.3. Hata Durumları                                                  | 18        |
| <b>3. Alt Düzey Tasarım</b>                                            | <b>18</b> |
| 3.1. Nesne Tasarım Kararları                                           | 18        |
| 3.1.1. Veritabanı Yerine JSON Tabanlı Dosya Sistemi Kullanımı (Revize) | 18        |
| 3.1.2. Varlık (Entity) ve Yönetici (Control/Manager) Sınıf Ayrımı      | 19        |
| 3.2. Son Nesne Tasarımı                                                | 20        |
| 3.3. Paketler / Katmanlar                                              | 20        |
| 3.3.1. Dış Paketler                                                    | 20        |
| 3.3.2. İç Paketler ve Mantıksal Modüller                               | 21        |
| 3.4. Sınıf Arayüzleri                                                  | 22        |
| 3.4.1. Giriş                                                           | 22        |
| 3.4.2. Arayüz (Boundary) Sınıfları                                     | 22        |
| 3.4.2.1. ErisimArayuzu                                                 | 23        |
| 3.4.2.2. AnaAkisArayuzu                                                | 23        |

|                                                 |           |
|-------------------------------------------------|-----------|
| 3.4.3. Yönetici (Control) Sınıfları             | 24        |
| 3.4.3.1. KimlikYönetici                         | 24        |
| 3.4.3.2. YorumYönetici                          | 24        |
| 3.4.3.3. FeedYönetici                           | 24        |
| 3.4.3.4. BeğeniYönetici                         | 25        |
| 3.4.3.5. GonderiYönetici                        | 25        |
| 3.4.4. Varlık (Entity) Sınıfları                | 26        |
| 3.4.4.1. Kullanıcı                              | 26        |
| 3.4.4.2. Feed(Akis)                             | 26        |
| 3.4.4.3. Gonderi                                | 27        |
| 3.4.4.4. Yorum                                  | 27        |
| 3.4.4.5. Beğeni                                 | 27        |
| 3.4.4.6. Oturum (Session)                       | 28        |
| 3.4.5. Veri Yönetimi ve Kalıcılık (Persistence) | 28        |
| 3.4.6. Yardımcı (Utility) Sınıf                 | 29        |
| 3.4.6.1. Jsonİsleyicisi                         | 29        |
| 3.4.7. Tasarım Kararları ve İlişkiler           | 29        |
| 3.5. Tasarım Desenleri                          | 30        |
| <b>Sözlük (Glossary)</b>                        | <b>32</b> |
| <b>Referanslar</b>                              | <b>34</b> |

# 1. Giriş

## 1.1. Sistemin Amacı

UniSocial, üniversite öğrencilerine özel olarak tasarlanmış web tabanlı bir sosyal ağ platformudur. Sistemin temel amacı, yalnızca .edu uzantılı kurumsal e-posta adresine sahip öğrencilerin kayıt olabileceği, güvenli ve kontrollü bir dijital sosyal ortam oluşturmaktır.

Sistem; kullanıcıların gönderi paylaşmasına, diğer kullanıcıları takip etmesine, paylaşımlara yorum yapmasına ve beğeni bırakmasına olanak tanımaktadır. Hedef kullanıcı kitlesi yalnızca üniversite öğrencilerinden oluşmakta olup platform, bu kullanıcıların birbirleriyle etkileşim kurabilecekleri sade ve işlevsel bir yapı sunmaktadır.

Sistem, istemci-sunucu mimarisine dayalı olarak geliştirilmekte; arka uç mantığı Python ve Flask framework'ü üzerinde yürütülmekte, veri yönetimi ise harici bir ilişkisel veritabanı kullanılmaksızın, veri kalıcılığını sağlayan JSON tabanlı dosya sistemi aracılığıyla gerçekleştirilmektedir. İşlemlerin performansı için çalışma zamanında Python'ın bellek içi veri yapıları (liste ve sözlük) kullanılsa da, yapılan tüm değişiklikler anlık olarak JSON dosyalarına senkronize edilerek güvenli bir şekilde saklanmaktadır.

## 1.2. Tasarım Hedefleri

Bu bölümde UniSocial sisteminin işlevsel olmayan gereksinimleri ve bu gereksinimlerin tasarım kararlarına nasıl yansıtıldığı açıklanmaktadır.

### 1.2.1. Güvenlik(Security)

UniSocial, yapısal olarak yalnızca üniversite öğrencilerine hitap eden kapalı devre bir platform olduğundan, sisteme erişim tamamen .edu ve .edu.tr uzantılı e-posta adresleriyle sınırlandırılmıştır. Kayıt aşamasında devreye giren Regex tabanlı e-posta doğrulama mekanizması, yetkisiz kullanıcıların sisteme dahil olmasını kesin olarak engellemektedir. Veri güvenliği kapsamında kullanıcı şifreleri veri dosyalarında (JSON) düz metin (plain-text) yerine hash'lenerek muhafaza edilmektedir. Uygulama genelinde oturum yönetimi, Flask framework'ünün sunduğu session mekanizması aracılığıyla kontrollü bir biçimde yürütülmektedir. Ayrıca; gönderi paylaşma, yorum yapma ve takip etme gibi kimlik doğrulama gerektiren tüm kritik sistem işlemlerinde aktif oturumun varlığı KimlikYoneticisi sınıfı tarafından sürekli olarak denetlenmektedir.

### 1.2.2. Kullanılabilirlik(Usability)

Sistem, son kullanıcıların herhangi bir teknik ön bilgiye ihtiyaç duymaksızın platformu verimli bir şekilde kullanabilmesi amacıyla sade, sezgisel ve anlaşılır bir arayüzle tasarlanmıştır. Kayıt olma, sisteme giriş, içerik paylaşımı, kullanıcı takibi ve profil

görüntüleme gibi temel işlevler, minimum kullanıcı etkileşimi (tıklama/adım) ile tamamlanacak şekilde optimize edilmiştir. Arayüz tasarımında navigasyon hızı ve erişilebilirlik ön planda tutulmuştur. Bununla birlikte, boş gönderi paylaşımı, geçersiz e-posta girişi veya hatalı şifre denemesi gibi istisnai kullanıcı hataları, sistem tarafından anlık olarak yakalanarak kullanıcıya doğrudan ve açıklayıcı geri bildirim (hata mesajları) sunulmaktadır. .

### **1.2.3. Bakım Yapılabilirlik ve Geliştirilebilirlik (Maintainability)**

Proje mimarisi, Nesne Yönelimli Programlama (OOP) prensipleri ve Katmanlı Mimari (Layered Architecture) yaklaşımı temel alınarak inşa edilmiştir. Sistemdeki sorumlulukların net bir biçimde ayrıştırılması amacıyla KimlikYonetici, GonderiYonetici, FeedYonetici ve YorumYonetici gibi özelleşmiş kontrol sınıfları benimsenmiştir. Bu yüksek kohezyon (cohesion) ve düşük bağımlılık (coupling) sunan modüler altyapı sayesinde; mesajlaşma veya anlık bildirim sistemi gibi ileride planlanılabilecek yeni özellikler, mevcut çekirdek bileşenlerin işleyişini bozmadan sisteme kolaylıkla entegre edilebilecektir.

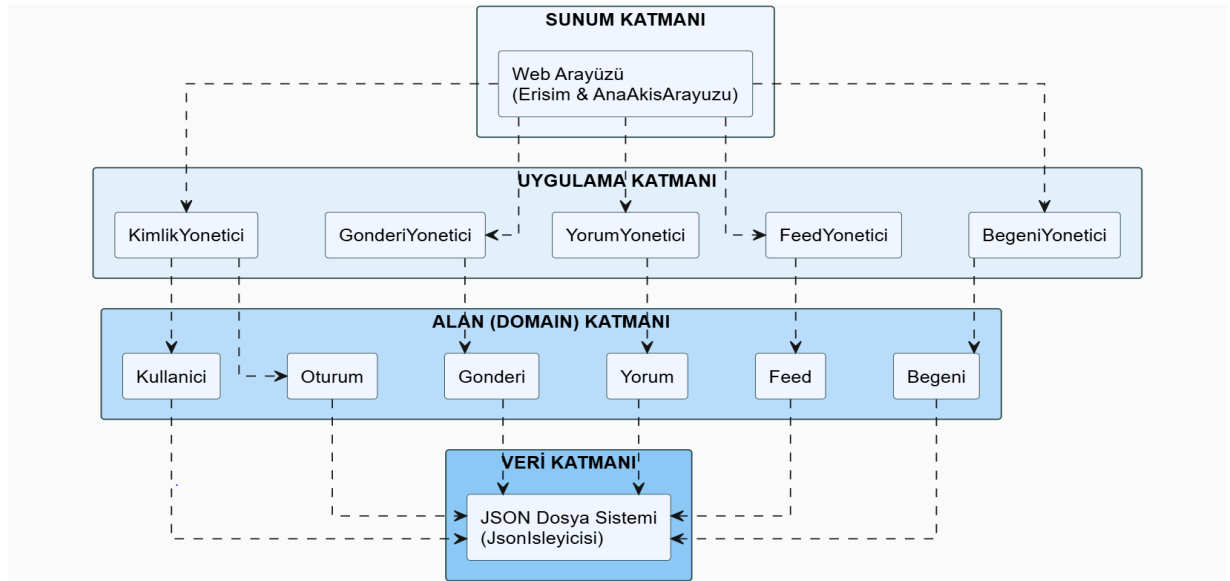
## 2. Üst Düzey Yazılım Mimarisi

Bu bölümde sistemin mimari yapısı, alt sistemlere ayrıştırma, donanım/yazılım eşlemesi, veri yönetimi, erişim kontrolü, kontrol akışı ve sınır koşulları ele alınır.

### 2.1. Alt Sistem Ayrıştırması

Sistemimiz; modülerliği artırmak, bakım süreçlerini kolaylaştırmak ve sorumlulukların net bir şekilde ayrılmasını sağlamak amacıyla Katmanlı Mimari prensibiyle tasarlanmıştır. Aşağıdaki şekilde gösterilen alt sistem ayrıştırması, sistemin hiyerarşik yapısını ve bağımlılık akışını temsil etmektedir. Sistemimiz 4 ana katmandan oluşmaktadır:

#### UniSocial - Katmanlı Mimari



**Sunum Katmanı:** Sistemin kullanıcı ile temas eden noktasıdır. **Web Arayüzü (Erisim & AnaAkisArayuzu)** bileşeni, kullanıcıdan gelen girdileri alır ve bunları işlenmek üzere Uygulama Katmanı'ndaki ilgili yöneticilere iletir. Bu katman, mimari izolasyon gereği doğrudan veri katmanı ile iletişim kurmaz.

**Uygulama Katmanı:** Sistemin iş mantığını (business logic) koordine eden merkezdir. Bu katmanda sorumlulukların ayrıştırılması amacıyla şu alt sistemler yer alır:

- **KimlikYonetici:** Kayıt, giriş ve oturum yönetimi süreçlerini yürütür.
- **GonderiYonetici:** Gönderi oluşturma, listeleme ve yetki bazlı silme işlemlerini yönetir.
- **YorumYonetici:** Paylaşılan içeriklere gelen yorumların veri bütünlüğünü ve yönetimini sağlar.

- **FeedYönetici:** Kullanıcının ana sayfasındaki içerik akışını ve sıralamasını organize eder.
- **BegeniYönetici:** İçerikler üzerindeki etkileşim ve beğeni süreçlerini kontrol eder.

**Alan (Domain) Katmanı:** Sistemin temel iş nesnelerini ve veri modellerini barındırır. **Kullanıcı, Gonderi, Yorum, Feed ve Beğeni** gibi varlıklar (entity) bu katmanda tanımlanır. Bu modeller, sistem genelinde tutarlı bir veri yapısı sunarak katmanlar arası veri transferini sağlar.

**Veri Katmanı:** Sistemin en alt katmanıdır. Tasarım hedefleri ve kurulum kolaylığı doğrultusunda, verilerin kalıcı olarak saklanması için **JSON Dosya Sistemi (JsonIsleyicisi)** yapısı tercih edilmiştir. Tüm üst katman nesneleri veri kalıcılığı ve erişimi için bu merkezi birimi kullanır.

## 2.2. Donanım/Yazılım Eşlemesi

UniSocial platformu, nesne yönelimli programlama prensiplerine dayalı, hafif ve taşınabilir bir web mimarisi olarak tasarlanmıştır. Sistem, istemci–sunucu (client-server) mimarisine uygun olarak çalışmakta olup, uygulamanın sunucu tarafı bileşenleri Python programlama dili ve Flask framework kullanılarak geliştirilmiştir.

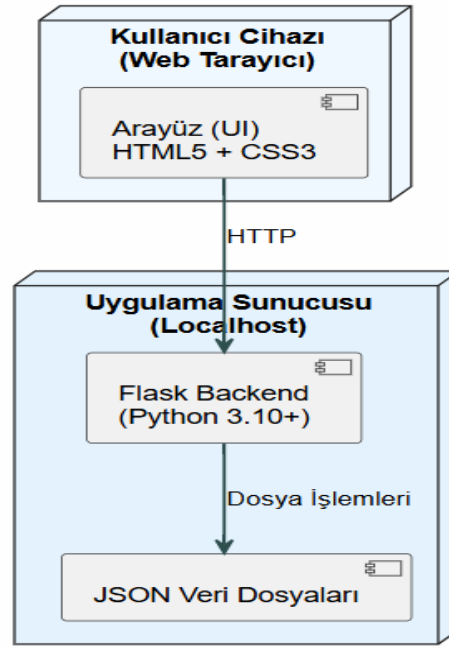
**Yazılım Bileşenleri:** Backend tarafında, HTTP isteklerinin yönetilmesi ve uygulama mantığının yürütülmesi amacıyla Flask (v3.0.x) framework’ü tercih edilmiştir. Programlama dili olarak Python (v3.10 ve üzeri), modüler yapısı ve güçlü veri işleme kabiliyetleri nedeniyle kullanılmıştır. Kullanıcı arayüzü (frontend) tarafında ise HTML5 ve CSS3 teknolojileri kullanılarak modern web standartlarına uygun, sunucu taraflı render edilen dinamik içeriklerin sunulabildiği bir yapı kurgulanmıştır.

**Veri Yönetimi ve Donanım Gereksinimleri:** Sistemde karmaşıklığı azaltmak ve kurulum maliyetini minimuma indirmek amacıyla harici bir ilişkisel veritabanı (MySQL, PostgreSQL vb.) kullanılmamaktadır. Bunun yerine, verilerin kalıcılığını sağlamak için JSON tabanlı bir dosya sistemi tercih edilmiştir. Veriler, çalışma zamanında Python veri yapıları aracılığıyla işlenmekte ve JsonIsleyicisi bileşeni vasıtasıyla yerel depolama birimindeki JSON dosyalarına aktarılmaktadır. Bu mimari yaklaşım sayesinde, yaklaşık 1 GB RAM ve 1 vCPU işlemci kapasitesine sahip standart bir sunucu veya kişisel bilgisayar, sistemin stabil çalışması için yeterlidir. Geliştirme ve test süreçlerinde yerel sunucu (localhost) kullanımı öngörülmektedir.

**İstemci Tarafı ve Uyumluluk:** Uygulama web tabanlı olduğu için herhangi bir özel donanım bileşeni gerektirmemektedir. Kullanıcıların modern bir web tarayıcısını çalıştırabilecek donanıma sahip olması erişim için yeterlidir. Platform; Google Chrome

(v90+), Mozilla Firefox (v88+), Microsoft Edge ve Safari (v14+) tarayıcıları ile tam uyumlu çalışacak şekilde tasarlanmıştır. Ancak kullanılan güncel web standartları nedeniyle Internet Explorer gibi eski nesil tarayıcılar desteklenmemektedir.

### UniSocial - Deployment Diagram



UniSocial sisteminin dağıtım mimarisi, istemci–sunucu modeline dayanmaktadır. İstemci tarafında kullanıcılar, uygulamaya web tarayıcısı üzerinden erişmekte ve HTML5 ile CSS3 kullanılarak oluşturulmuş kullanıcı arayüzü ile etkileşime geçmektedir.

Sunucu tarafında ise Flask framework kullanılarak geliştirilmiş backend uygulaması çalışmaktadır. Bu bileşen, istemciden gelen HTTP isteklerini işleyerek sistemin iş mantığını yürütmekte ve gerekli veri işlemlerini gerçekleştirmektedir.

Veri yönetimi, harici bir veritabanı yerine JSON tabanlı dosya sistemi aracılığıyla sağlanmaktadır. Backend bileşeni, verileri çalışma zamanında işleyerek yerel depolama üzerindeki JSON dosyalarına aktarmaktadır.

İstemci ile sunucu arasındaki iletişim HTTP protokolü üzerinden gerçekleşmektedir.

### 2.3. Kalıcı Veri Yönetimi

UniSocial sistemi, eğitim amaçlı geliştirilen web tabanlı bir sosyal ağ platformudur. Bu kapsamda sistemde, geleneksel bir veritabanı yönetim sistemi (RDBMS) yerine sistemin



ölçeğine ve amacına uygun olan JSON (JavaScript Object Notation) tabanlı veri saklama yaklaşımı tercih edilmiştir.

### 2.3.1. Kalıcı Olarak Saklanacak Veriler

Sistem kapsamında, kullanıcıların platform üzerindeki etkileşimlerini kalıcı hale getirmek amacıyla aşağıdaki temel veri grupları saklanmaktadır:

- **Kullanıcı Bilgileri:** Kullanıcı kimliği (ID), kullanıcı adı, e-posta adresi, şifre, bölüm ve profil bilgileri.
- **Gönderiler:** Gönderi kimliği (ID), gönderi içeriği, oluşturulma zamanı (zaman damgası) ve gönderi sahibine ait kullanıcı ID'si.
- **Yorumlar:** Yorum kimliği (ID), yorum metni, oluşturulma zamanı (zaman damgası), ilgili gönderinin ID'si ve yorumu yapan kullanıcının ID'si.
- **Beğeniler:** Beğeni kimliği (ID), beğenilen gönderinin ID'si ve beğeniyi yapan kullanıcının ID'si.
- **Takip İlişkileri:** Takip eden kullanıcı ID'si ve takip edilen kullanıcı ID'sini içeren sosyal bağlantı verileri.

### 2.3.2. Veritabanı Teknolojisi ve Seçim Gerekçeleri

Proje kapsamında veri saklama teknolojisi olarak **JSON dosyaları** kullanılmaktadır. Bu teknolojinin tercih edilmesinin temel gerekçeleri şunlardır:

- **Eğitim Odaklılık:** Eğitim amaçlı geliştirilen bu projede, harici bir veritabanı kurulumunun (SQL/NoSQL) gereksiz bir kurulum karmaşıklığı yaratmasını önlemek.
- **Kolay Entegrasyon:** Verilerin Python programlama dilindeki gömülü json kütüphanesi ile list ve dictionary (sözlük) yapılarına kolayca dönüştürülüp işlenebilmesi.
- **Okunabilirlik:** JSON formatının insan tarafından kolayca okunabilir ve gerektiğinde manuel olarak düzenlenebilir, şeffaf bir yapıda olması.
- **Taşınabilirlik:** Hafif yapısı sayesinde sistemin taşınabilir olması ve küçük ölçekli projelerde veri kalıcılığı için son derece uygun performans göstermesi.

*Not: JSON tabanlı bu yaklaşım, hızlı geliştirme imkanı sunsa da eş zamanlı erişim durumlarında veri çakışması riski taşıyabilmektedir. Bu nedenle sistemin ileride büyütülmesi veya çok kullanıcıya geniş ölçekli bir yapıya bürünmesi durumunda, daha gelişmiş ölçeklenebilir veritabanı sistemlerine geçiş yapılması önerilmektedir.*

### 2.3.3. Veri Eriřim Stratejisi

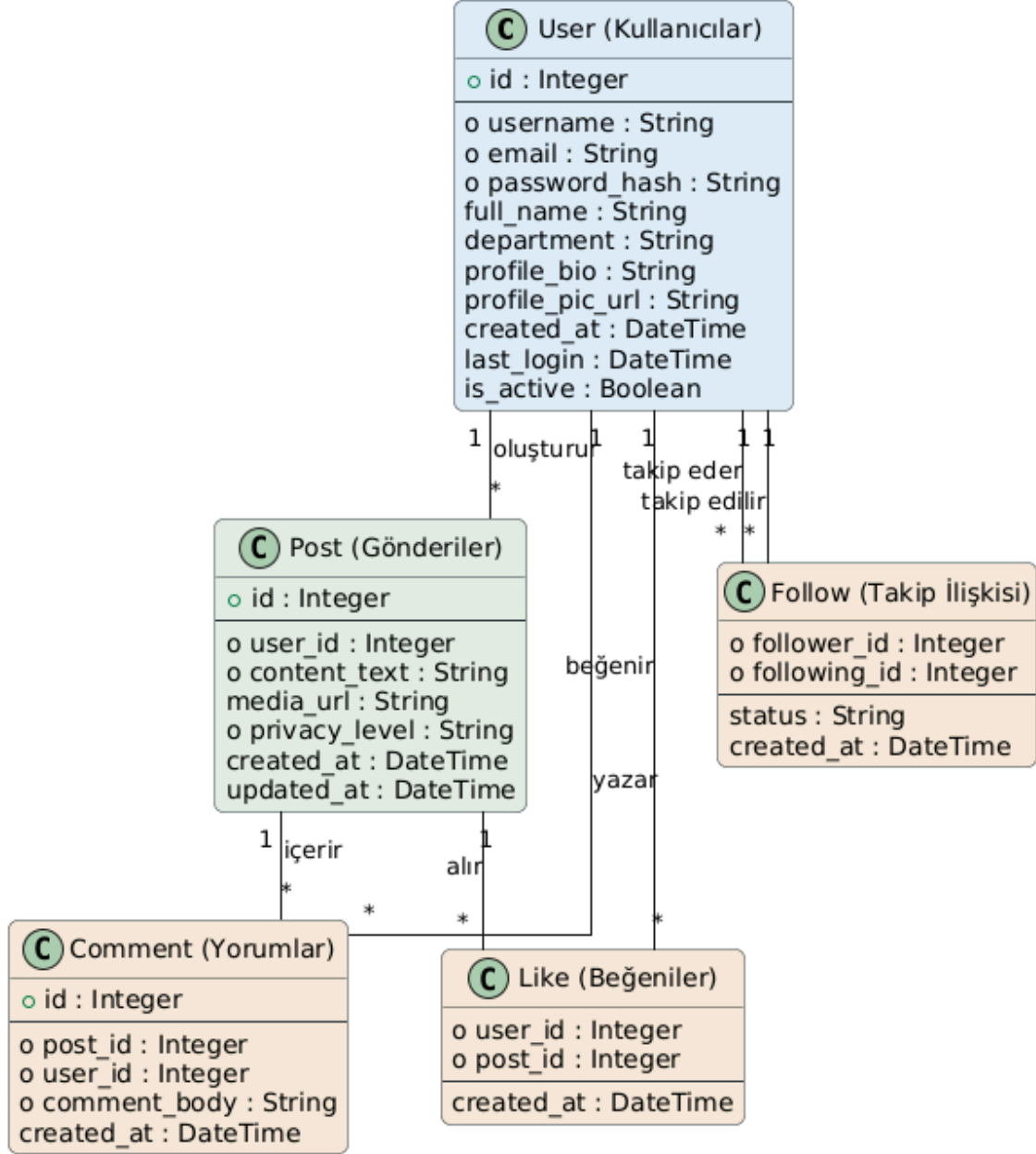
UniSocial sisteminde veri eriřimi, katmanlı mimari prensiplerine uygun olarak doğrudan kullanıcı arayüzü üzerinden değil, iş mantığını yöneten **kontrol (controller)** katmanı üzerinden gerçekleştirilmektedir. Veri eriřim işlemleri, JSON verileri ile bellek arasında veri akışını sağlayan yardımcı sınıflar aracılığıyla gerçekleştirilmektedir. Arayüz yalnızca veri girişini almak ve sonuçları kullanıcıya sunmakla sorumludur. Veri eriřim süreci řu stratejilere dayanır:

- **Bellek ve JSON Senkronizasyonu:** Sistem çalışırken veriler performans optimizasyonu amacıyla Python *list* ve *dictionary* yapılarında geçici bellekte (RAM) tutulur. Okuma (Read) işlemleri yüksek hız sağlamak için bu bellek üzerinden yapılır.
- **Kalıcılığın Sağlanması:** Sistem üzerinde gerçekleştirilen her CRUD (Oluşturma, Okuma, Güncelleme, Silme) işlemi sonrasında, bellekteki güncel veri yapıları anında ilgili JSON dosyalarına yazılarak kalıcılık ve veri tutarlılığı sağlanır. Bu durum gereksiz dosya erişimlerini azaltarak sistemi hızlandırır.
- **Modüler Dosya Yönetimi:** Veri karmaşıklığını önlemek amacıyla veriler yapılarına göre ayrıştırılmış JSON dosyalarında yönetilir:
  - Kullanıcı kayıt ve ekleme işlemleri ->kullanıcılar .json
  - Gönderi oluşturma ve paylaşma işlemleri ->gönderiler .json
  - Yorum ekleme işlemleri ->yorumlar.json
  - Beğeni etkileşimleri ->begeniler.json
  - Takip etme/bırakma işlemleri ->takipler.json

Bu sıralı işlem yapısı; verilerin okunması, iş mantığı kurallarının bellekte uygulanması ve son olarak güncellenmiş verinin kalıcı diske yazılması döngüsüyle veri bütünlüğünün korunmasını sağlamaktadır.

### 2.3.4. ER Diyagramı

#### UniSocial - ER Diyagramı



### 2.4. Erişim Kontrolü ve Güvenlik

UniSocial platformu, sadece üniversite topluluğuna özel bir yapı sunduğundan, güvenliğin temelini "hedeflenmiş kullanıcı kitlesi" ve "içerik sahipliği" ilkeleri oluşturmaktadır. Sistem, verilerin gizliliğini sağlamak ve yetkisiz müdahaleleri önlemek adına kimlik doğrulama ve yetkilendirme süreçlerini katı kurallarla yönetmektedir.

#### **2.4.1. Erişim Kontrol Matrisi**

Sistemin sunduğu işlevler, kullanıcıların oturum durumuna göre belirli yetkilendirme seviyelerine ayrılmıştır. Aşağıdaki matris, platformdaki farklı kullanıcı tiplerinin hangi rollere ve yetkilere sahip olduğunu göstermektedir:

| İşlev / Yetki                      | Ziyaretçi | Doğrulanmış Öğrenci | İçerik Sahibi | Yönetici (Admin) |
|------------------------------------|-----------|---------------------|---------------|------------------|
| Ana Sayfayı Görüntüleme            | X         | X                   | X             | X                |
| Kayıt Olma                         | X         |                     |               | X                |
| Giriş Yapma                        | X         | X                   | X             | X                |
| Çıkış Yapma                        |           | X                   | X             | X                |
| Şifre Sıfırlama                    | X         | X                   | X             | X                |
| E-posta Doğrulama (.edu / .edu.tr) |           | X                   | X             | X                |
| Profil Görüntüleme                 |           | X                   | X             | X                |
| Profil Düzenleme                   |           | X                   | X             | X                |
| Kullanıcı Arama                    |           | X                   | X             | X                |
| Kullanıcı Takip Etme               |           | X                   | X             | X                |
| Takibi Bırakma                     |           | X                   | X             | X                |
| Akış (Feed) Görüntüleme            |           | X                   | X             | X                |
| Gönderi Oluşturma                  |           | X                   | X             | X                |

| İşlev / Yetki              | Ziyaretçi | Doğrulanmış Öğrenci | İçerik Sahibi | Yönetici (Admin) |
|----------------------------|-----------|---------------------|---------------|------------------|
| Gönderi Düzenleme          |           |                     | X             | X                |
| Gönderi Silme              |           |                     | X             | X                |
| Gönderi Beğenme            |           | X                   | X             | X                |
| Beğeniye Geri Alma         |           | X                   | X             | X                |
| Yorum Yapma                |           | X                   | X             | X                |
| Yorumu Silme               |           |                     | X             | X                |
| Hashtag ile Arama          |           | X                   | X             | X                |
| Kullanıcı Raporlama        |           | X                   | X             | X                |
| İçerik Denetleme           |           |                     |               | X                |
| Kullanıcı Askıya Alma      |           |                     |               | X                |
| Hesap Kapatma              |           | X                   | X             | X                |
| Oturum Doğrulama (Session) |           | X                   | X             | X                |
| Yetkilendirme Kontrolü     |           | X                   | X             | X                |

| İşlev / Yetki          | Ziyaretçi | Doğrulanmış Öğrenci | İçerik Sahibi | Yönetici (Admin) |
|------------------------|-----------|---------------------|---------------|------------------|
| Sistem Log Görüntüleme |           |                     |               | X                |

#### 2.4.2. Kimlik Doğrulama (Authentication) Süreçleri

Kullanıcıların sisteme kabul edilmesi ve kimliklerinin doğrulanması aşamasında aşağıdaki teknik prosedürler izlenmektedir:

- **Akademik E-posta Zorunluluğu:** Platformun kapalı devre kalmasını sağlamak adına kayıt aşamasında yalnızca “.edu” ve “.edu.tr” uzantılı e-posta adreslerine izin verilmektedir. Bu kontrol, veri giriş aşamasında **Düzenli İfadeler (Regex)** kullanılarak gerçekleştirilmektedir.
- **Kimlik Doğrulama :** Giriş işlemleri sırasında kullanıcıdan alınan kullanıcı adı ve şifre bilgileri, sistemde kayıtlı mevcut verilerle karşılaştırılmaktadır. Bilgilerin eşleşmesi durumunda sisteme erişim izni verilirken, hatalı girişlerde kullanıcı bilgilendirici bir hata mesajı ile tekrar giriş ekranına yönlendirilmektedir.
- **Oturum (Session) Yönetimi:** Başarılı bir girişin ardından sunucu tarafında kullanıcıya özel bir oturum değişkeni oluşturulmaktadır. Kullanıcı sistemden çıkış yaptığında bu oturum verileri tamamen temizlenmekte ve aktif erişim sonlandırılmaktadır.
- **Şifre Güvenliği:** Veritabanında (JSON) veri güvenliğini sağlamak adına şifreler düz metin (plain text) olarak saklanmamakta; **hash algoritmaları** kullanılarak korunmaktadır.

#### 2.4.3. Yetkilendirme (Authorization) ve Erişim Sınırları

Sisteme dahil olan kullanıcıların platform üzerindeki yetkileri, veri bütünlüğünü korumak amacıyla belirli sınırlar dahilinde tutulmaktadır:

- **İşlevsel Yetkilendirme:** Gönderi paylaşma, yorum yapma, beğeni bırakma ve takipleşme gibi etkileşimli özellikler yalnızca aktif bir oturumu bulunan "Doğrulanmış Öğrenci" rolündeki kullanıcılara açıktır.
- **İçerik Sahipliği Kısıtlaması:** Kullanıcılar sistemdeki tüm içerikleri görüntüleme yetkisine sahip olsalar da düzenleme ve silme yetkileri yalnızca **kendilerine ait içeriklerle** (kendi gönderileri veya kendi yorumları) sınırlandırılmıştır. Başka bir kullanıcıya ait verilerin modifiye edilmesi sistem mimarisi tarafından engellenmektedir.
- **İşlem Denetimi:** Her CRUD (oluşturma, okuma, güncelleme, silme) işlemi öncesinde sistem, işlemi gerçekleştirmeye çalışan kullanıcının oturum yetkilerini ve içeriğin sahibi olup olmadığını kontrol etmektedir.

#### 2.4.4. Veri Güvenliği ve Alınan Ek Önlemler

Sistemin sürdürülebilirliğini ve kullanıcı verilerinin gizliliğini artırmak için teknik düzeyde ek güvenlik katmanları uygulanmaktadır:

- **Girdi Doğrulaması:** Boş veri girişlerinin engellenmesi, kullanıcı bilgileri için benzersizlik denetimleri (username/email) ve şifreler için minimum uzunluk kontrolleri sistem genelinde uygulanmaktadır.
- **JSON Dosya Koruması:** Kalıcı veri dosyaları (JSON), yetkisiz doğrudan erişimlere karşı korunmakta ve her işlem sonrası veriler güvenli bir şekilde senkronize edilmektedir.
- **Gizlilik Politikası:** Kullanıcı verileri sadece platform içi işlemler için kullanılmakta ve kesinlikle üçüncü taraflarla paylaşılmamaktadır.
- **Otomatik Oturum Kapatma:** Güvenlik risklerini minimize etmek adına, belirli bir süre boyunca hiçbir işlem yapılmayan (idle) oturumların sistem tarafından otomatik olarak sonlandırılması hedeflenmektedir.
- **Modüler Mimari:** Kod yapısının modüler ve sürdürülebilir bir tasarıma sahip olması, olası güvenlik açıklarının hızlıca tespit edilmesini ve güncellenmesini kolaylaştırmaktadır.

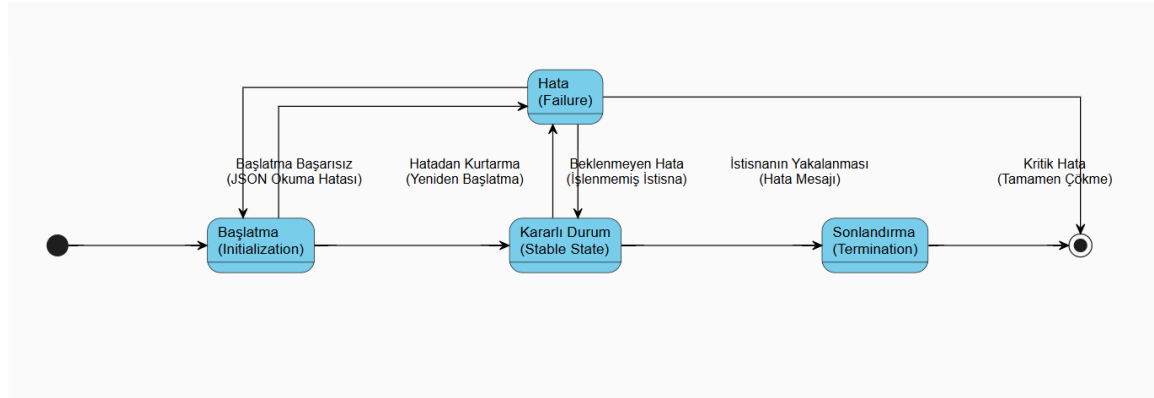
#### 2.5. Küresel Kontrol Akışı

UniSocial, olay güdümlü (event-driven) ve senkron bir kontrol akışı modelini benimsemektedir. Kullanıcı arayüzünden gelen her istek, Flask'ın yönlendirme (routing) mekanizması aracılığıyla ilgili yönetici sınıfına (Controller) iletilmekte; bu sınıf gerekli



işlemleri gerçekleştirerek sonucu katmanlı yapı üzerinden arayüze geri döndürmektedir. Sistemin kontrol akışı üç ana katman üzerinden yürütülmektedir. Sunum Katmanı (Presentation Layer), web tarayıcısı üzerinde çalışan HTML/CSS arayüzlerinden oluşmakta ve kullanıcı eylemlerini Flask route'larına iletmektedir. Uygulama Katmanı (Application Layer), KimlikYonetici, GonderiYonetici, FeedYonetici ve YorumYonetici sınıflarını barındırmakta; iş mantığını bu yönetici sınıflar üzerinden yürütmektedir. Veri Katmanı (Data Layer) ise, Kullanici, Gonderi, Yorum ve Begeni nesnelerini kalıcı olarak saklamak üzere JSON tabanlı dosya sistemi kullanmaktadır. Sistem performansı için veriler çalışma zamanında liste ve sözlük gibi yapılarda işlense de, gerçekleştirilen her işlemin ardından güncel veriler JsonIsleyicisi aracılığıyla anlık olarak JSON dosyalarına aktarılarak kalıcılık sağlanır ve sistemde harici bir ilişkisel veritabanı kullanılmaz.

## 2.6. Sınır Koşulları



### 2.6.1. Başlatma

UniSocial sistemi bir web uygulaması olarak, sunucu tarafının başlatılması ile çalışır hale gelmektedir. Uygulama geliştirme ortamında Flask'ın yerleşik sunucusu üzerinden başlatılmakta olup, gerektiğinde Gunicorn gibi bir WSGI sunucusu ile çalıştırılabilir şekilde tasarlanmıştır. Sistem başlatıldığında herhangi bir harici SQL veritabanı bağlantısı kurulmamaktadır. Bunun yerine, sistem başlatılırken JsonIsleyicisi modülü devreye girer ve yerel dizindeki mevcut JSON dosyalarından (kullanici.json, gonderiler.json vb.) daha önceden kaydedilmiş kalıcı verileri okuyarak sistem belleğine yükler. Eğer ilgili dosyalar henüz yoksa, sistem başlangıç durumu olarak bu dosyaları boş bir şekilde otomatik oluşturur. Başlatma sürecinde uygulamaya ait tüm route tanımları yüklenmekte ve oturum yönetimi aktif hale getirilmektedir. Kullanıcı arayüzü başlangıçta giriş/kayıt

ekranına yönlendirilmekte ve sisteme erişim yalnızca kimlik doğrulamasından sonra mümkün olmaktadır.

### 2.6.2. Sonlandırma

UniSocial sisteminin sonlandırılması kullanıcı kaynaklı ve sunucu kaynaklı olmak üzere iki farklı biçimde gerçekleşmektedir.

Kullanıcı kaynaklı sonlandırma, oturum açmış bir kullanıcının sistemden çıkış yapması ile gerçekleşir. Kullanıcı "Çıkış Yap" butonuna bastığında, KimlikYonetici.oturumSonlandir() metodu çağrılmakta, Flask oturum (session) nesnesi temizlenmekte ve kullanıcı güvenli bir şekilde giriş ekranına yönlendirilmektedir. Bu süreçte aktif oturum kapatılarak yetkisiz erişimlerin önüne geçilmektedir.

Sunucu kaynaklı sonlandırma ise uygulamanın çalıştığı sunucunun durdurulması ile gerçekleşir. Sistemde veri kalıcılığı anlık olarak senkronize edilen JSON dosyaları aracılığıyla sağlandığından, sunucu kapatıldığında veya yeniden başlatıldığında kullanıcı verilerinde hiçbir kayıp yaşanmaz. Tüm etkileşimler dosyaya yazılı kalmaya devam ederken yalnızca geçici oturum (session) verileri sonlandırılmış olur.

### 2.6.3. Hata Durumları

Sistem, öngörülebilir hata senaryolarını uygulama ve veri katmanında ele almaktadır.

Veri erişimi ve kalıcılık işlemleri sırasında ortaya çıkabilecek JSON dosya okuma/yazma sorunları JsonIsleyicisi tarafından yakalanarak sistem kararlılığı korunur.

Geçersiz giriş verileri durumunda (hatalı şifre, kayıtlı olmayan e-posta), KimlikYonetici ilgili hata mesajını oluşturmakta ve kullanıcıyı giriş formuna geri yönlendirmektedir. Kayıt sırasında .edu uzantısı taşımayan e-posta adresleri Regex denetiminden geçemeyeceğinden işlem durdurulmakta, kullanıcıya "Üniversite maili gerekli" uyarısı gösterilmektedir.

Boş içerik gönderimi durumunda (boş gönderi metni, boş yorum) GonderiYonetici ve YorumYonetici sınıfları içerik doğrulamasını gerçekleştirmekte; geçersiz içerik tespit edildiğinde işlem iptal edilerek kullanıcıya bilgilendirici bir hata mesajı iletilmektedir.

Yetki ihlali durumunda, bir kullanıcı başka bir kullanıcıya ait gönderi veya yorumu silmeye ya da düzenlemeye çalıştığında sistem sahiplik doğrulaması yaparak işlemi reddetmekte ve yetkisiz erişim hatası döndürmektedir. Oturum doğrulaması gerektiren sayfalara oturum açılmadan erişilmeye çalışıldığında ise KimlikYonetici kullanıcıyı otomatik olarak giriş sayfasına yönlendirmektedir.

### 3. Alt Düzey Tasarım

UniSocial platformunun tasarım sürecinde, projenin "nesne yönelimli programlama" (OOP) hedefleri ve sistemin karmaşıklık düzeyi göz önüne alınarak bazı stratejik mimari kararlar alınmıştır. Bu kararlar ve sistemin genel yapısına etkileri (ödünleşimler/trade-offs) aşağıda detaylandırılmıştır:

#### 3.1. Nesne Tasarım Kararları

##### 3.1.1. JSON Tabanlı Dosya Sistemi Kullanımı

Sistemde verilerin kalıcı olarak saklanması amacıyla, geleneksel bir veritabanı yönetim sistemi (örneğin MySQL ve ya PostgreSQL) ile JSON tabanlı dosya sistemi arasında bir tercih yapılmıştır. Bu iki yaklaşım; performans, kurulum maliyeti ve sistem karmaşıklığı açısından karşılaştırılmıştır.

İlişkisel veritabanları, yüksek performans, gelişmiş sorgulama yetenekleri ve eşzamanlılık yönetimi gibi avantajlar sunmaktadır. Ancak bu sistemler, ek kurulum gereksinimleri, yapılandırma maliyeti ve proje karmaşıklığını artırması gibi dezavantajlara sahiptir.

Buna karşılık JSON tabanlı dosya sistemi, herhangi bir harici bağımlılık gerektirmemesi, kolay kurulumu ve insan tarafından okunabilir veri yapısı sayesinde geliştirme sürecini hızlandırmaktadır.

Bu bağlamda, UniSocial platformunun hedeflenen ölçeği ve eğitim odaklı yapısı göz önünde bulundurularak, bakım kolaylığı ve düşük sistem karmaşıklığı, yüksek performans gereksinimlerine göre daha öncelikli kabul edilmiş ve JSON tabanlı çözüm tercih edilmiştir.

##### 3.1.2. Varlık (Entity) ve Yönetici (Control/Manager) Sınıf Ayrımı

Sistemdeki sınıflar, Sınır-Kontrol-Varlık (Boundary-Control-Entity, BCE) tasarım deseni esas alınarak modellenmiştir. Bu kapsamında, yalnızca veri tutma sorumluluğuna sahip varlık sınıfları (örneğin Kullanici, Gonderi) ile bu veriler üzerinde iş mantığını yürüten yönetici sınıflar (örneğin KimlikYoneticisi, GonderiYoneticisi) birbirinden ayrılmıştır.

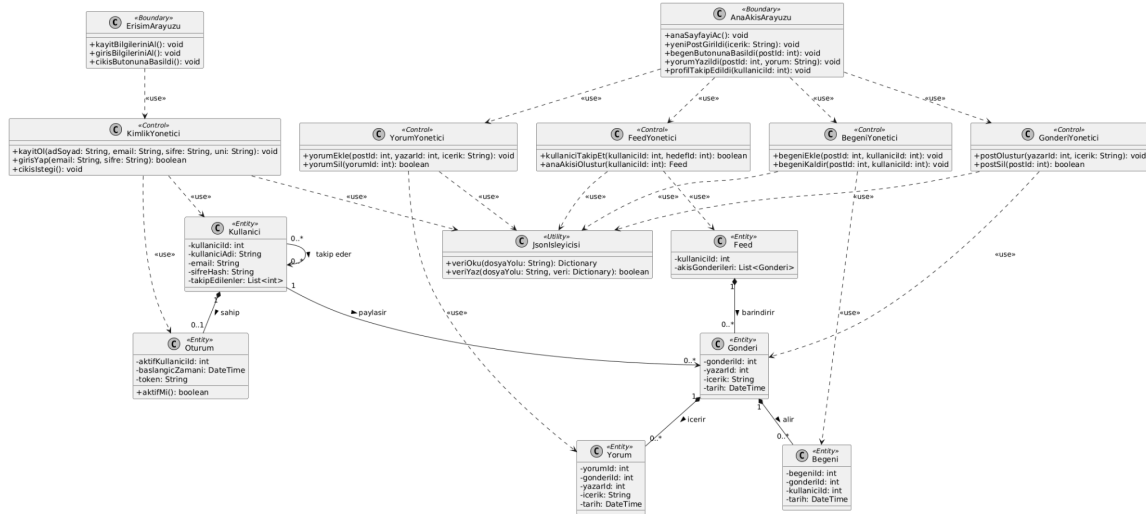
Bu ayrımın temel avantajı, sistemde yüksek içsel tutarlılık (high cohesion) ve düşük bağımlılık (low coupling) elde edilmesini sağlamasıdır. Böylece kodun okunabilirliği ve sürdürülebilirliği artmış, aynı zamanda test edilebilirlik önemli ölçüde iyileştirilmiştir. Ayrıca arayüz (boundary), iş mantığı (control) ve veri katmanlarının birbirinden izole edilmesi, katmanlı mimari prensiplerle uyumlu bir yapı ortaya koymaktadır.

Öte yandan, bu yaklaşım belirli bir mimari karmaşıklık maliyeti doğurmaktadır. Sınıfların sorumluluklara göre ayrıştırılması, toplam sınıf ve dosya sayısını artırarak başlangıç aşamasında sistemin anlaşılmasını zorlaştırabilmektedir. Ayrıca basit bir veri ekleme işlemi dahi birden fazla katman arasında etkileşim gerektirdiğinden, işlem akışı daha dolaylı hale gelmektedir.

Alternatif olarak, tüm işlevlerin tek bir sınıf içerisinde toplandığı daha basit bir tasarım yaklaşımı değerlendirilebilirdi. Ancak bu yaklaşım, uzun vadede bakım zorlukları ve düşük modülerlik gibi sorunlara yol açabileceğinden tercih edilmemiştir.

### 3.2. Son Nesne Tasarımı

#### UniSocial - Varlık (Entity) Diyagramı



Sistemin tüm sınıflarını, özneliklerini, metotlarını ve aralarındaki ilişkileri gösteren ayrıntılı UML sınıf diyagramı bu bölüme eklenir. Analiz raporundaki sınıf diyagramına göre daha detaylıdır ve tasarım sınıflarını da içerir.

### 3.3. Paketler / Katmanlar

UniSocial projesinin dizin ve paket yapısı, modülerliği en üst düzeye çıkarmak ve kodun okunabilirliğini artırmak amacıyla, belirlenen Katmanlı Mimari (Layered Architecture) ve MVC (Model-View-Controller) prensiplerine sıkı sıkıya bağlı kalınarak tasarlanmıştır. Bu yapı, hem bağımlılıkların yönetilmesini kolaylaştırmakta hem de takım içindeki paralel geliştirme süreçlerini desteklemektedir. Uygulama, işlevsel sorumluluklarına göre mantıksal paketlere (modüllere) ayrılmıştır.

#### 3.3.1. Dış Paketler ve Kütüphaneler

UniSocial platformunun temel iskeletini oluşturmak ve standart web işlemlerini güvenli bir şekilde yönetmek amacıyla, dış kaynaklı kütüphanelerden yararlanılmıştır. Bu kütüphaneler, uygulamanın yalnızca belirli katmanlarında görev alacak şekilde izole edilmiştir:

- **Flask (v3.0.x):**
  - **Kullanıldığı Katman:** Sunum (Presentation) ve Uygulama (Controller) Katmanları.
  - **Rolü:** Web uygulamasının temel HTTP yönlendirme (routing) işlemlerini, istek/yanıt döngülerini yönetmek amacıyla kullanılmıştır. Blueprint yapısı sayesinde uygulamanın modüler rotalara bölünmesini sağlar.
- **Werkzeug.security (Flask'ın Dahili Bağımlılığı):**
  - **Kullanıldığı Katman:** Uygulama (Controller) Katmanı (KimlikYonetici sınıfı).
  - **Rolü:** Kullanıcı şifrelerinin güvenli bir şekilde şifrelenmesi (hashing) ve doğrulanması işlemleri için kullanılmıştır. Şifrelerin düz metin (plaintext) olarak saklanması önüne geçer.
- **JSON (Python Standart Kütüphanesi):**
  - **Kullanıldığı Katman:** Veri (Data) Katmanı (JsonIsleyicisi sınıfı).
  - **Rolü:** Veritabanı olmaması kısıtlamasına çözüm olarak, verilerin fiziksel dosyalara yazılması ve bu dosyalardan Python sözlüklerine dönüştürülmesi sürecini üstlenir.
- **Datetime / UUID (Python Standart Kütüphaneleri):**
  - **Kullanıldığı Katman:** Alan (Domain/Entity) Katmanı.
  - **Rolü:** Gönderilerin, yorumların ve beğenilerin zaman damgalarını (timestamp) oluşturmak ve benzersiz kimlik numaraları (ID) üretmek amacıyla kullanılır.

### 3.3.2. İç Paketler ve Mantıksal Modüller

Uygulamanın iç yapısı, iş mantığını ve alan nesnelerini düzenli bir şekilde gruplamak için kendi içinde alt modüllere ayrılmıştır:

- **models/ (Alan/Entity Katmanı Modülü)**
  - **Sorumluluk:** Sistemin veri yapılarını ve bu verilerin kendi içsel doğrulamalarını barındırır. Bu dizin altındaki sınıflar doğrudan iş mantığı çalıştırmaz, veri depolama birimi ile iletişim kurmaz; yalnızca veriyi temsil eder.
  - **İçerik:** Kullanici.py, Gonderi.py, Yorum.py, Begeni.py, Feed.py, Oturum.py dosyaları bu pakette yer alır.
- **controllers/ (Uygulama/Control Katmanı Modülü)**
  - **Sorumluluk:** Sistemdeki ana iş mantığının yürütüldüğü, verilerin işlendiği ve nesneler arası koordinasyonun sağlandığı modüldür. Veri manipülasyonu ve kuralların işletilmesi tamamen bu katmanın sorumluluğundadır.
  - **İçerik:** KimlikYonetici.py, GonderiYonetici.py, FeedYonetici.py, YorumYonetici.py, BegeniYonetici.py dosyalarını içerir.
- **views/ veya routes/ (Sunum/Boundary Katmanı Modülü)**
  - **Sorumluluk:** Kullanıcı arayüzleri ile arka uç arasındaki köprüdür. HTTP üzerinden gelen istekleri karşılar, uygun Controller metodunu çağırır ve HTML şablonlarını döndürür.
  - **İçerik:** ErisimArayuzu.py ve AnaAkisArayuzu.py yapılarını barındırır.
- **utils/ (Yardımcı/Utility Katmanı Modülü)**
  - **Sorumluluk:** Uygulamanın farklı katmanlarında tekrar tekrar kullanılacak, iş mantığından bağımsız genel hizmetleri sunar.
  - **İçerik:** JsonIsleyicisi.py ve VeriDeposu.py sınıflarını içerir.
- **data/ (Kalıcı Veri Depolama Dizin Modülü)**
  - **Sorumluluk:** Çalışma zamanında üretilen verilerin kalıcı olarak saklandığı fiziksel JSON dosyalarının tutulduğu dizindir.
  - **İçerik:** kullanicilar.json, gonderiler.json, yorumlar.json vb. veri dosyalarını barındırır.
- **static/ ve templates/ (Statik Kaynaklar ve Arayüz Şablonları)**
  - **Sorumluluk:** Son kullanıcıya gösterilecek olan görsel ve arayüz dosyalarını barındırır.
  - **İçerik:** static/ dizini altında CSS ve JS dosyaları; templates/ dizini altında ise Flask'ın derleyeceği .html dosyaları yer alır.

### 3.4. Sınıf Arayüzleri

Her bir önemli sınıfın (entity, control, boundary) öznelikleri ve metotları detaylı olarak açıklanır. Her metot için imza (parametreler, dönüş değeri) ve kısa bir açıklama verilir.

#### 3.4.1. Giriş

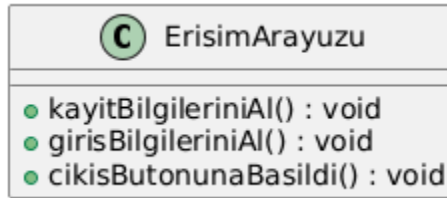
Bu dokümanda, geliştirilen web tabanlı sosyal ağ sistemine (UniSocial) ait yazılım mimarisi kapsamında kullanılan tüm sınıflar detaylı olarak açıklanmaktadır. Sistem; Boundary (Arayüz), Control (Yönetici) ve Entity (Varlık) katmanlarına uygun şekilde, nesne yönelimli programlama (OOP) prensipleriyle tasarlanmıştır. Bu yapı sayesinde sistem modüler hale getirilmiş, bakımı kolaylaştırılmış ve genişletilebilir bir mimari oluşturulmuştur.

#### 3.4.2. Arayüz (Boundary) Sınıfları

Bu katman, kullanıcının web sitesi ile girdiği etkileşimleri yakalar ve iş mantığını yürütmesi için ilgili Control (Yönetici) sınıflarına iletir.

##### 3.4.2.1. ErisimArayuzu

- Açıklama: Kullanıcının sisteme giriş ve kayıt işlemlerini gerçekleştirdiği arayüzdür. Form verilerini toplar ve KimlikYonetici sınıfına istek yollar.
- Metotlar:
  - + kayıtBilgileriniAl() : void -> Kullanıcıdan kayıt form verilerini alır.
  - + girişBilgileriniAl() : void -> Kullanıcıdan giriş form verilerini alır.
  - + cikisButonunaBasildi() : void -> Çıkış işlemini tetikler.



##### 3.4.2.2 AnaAkisArayuzu

- Açıklama: Kullanıcının ana sayfa (feed) üzerinde yaptığı işlemleri yönetir. Etkileşimleri ilgili yönetici sınıflara (YorumYonetici, FeedYonetici, GonderiYonetici, BeğeniYonetici) iletir.
- Metotlar:
  - + anaSayfayiAc() : void -> Ana akışı başlatır.
  - + yeniPostGirildi(icerik: String) : void -> Gönderi içeriğini alır.
  - + begenButonunaBasildi(postId: int) : void -> Beğeni isteğini yakalar.
  - + yorumYazildi(postId: int, yorum: String) : void -> Yorum metnini alır.

- + profilTakipEdildi(kullaniciId: int) : void -> Takip isteğini iletir.

| <b>C</b> AnaAkisArayuzu                                                                                                                                                                                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• anaSayfayiAc() : void</li> <li>• yeniPostGirildi(icerik : String) : void</li> <li>• begenButonunaBasildi(postId : int) : void</li> <li>• yorumYazildi(postId : int, yorum : String) : void</li> <li>• profilTakipEdildi(kullaniciId : int) : void</li> </ul> |

### 3.4.3. Yönetici (Control) Sınıfları

Arayüzden gelen istekleri işleyen ve Varlık (Entity) sınıflarını yöneten iş mantığı katmanıdır.

#### 3.4.3.1 KimlikYoneticisi

- Açıklama: Kullanıcı doğrulama ve hesap işlemlerini üstlenir.
- Metotlar:
  - + kayitOl(adSoyad: String, email: String, sifre: String, uni: String) : void  
-> Yeni kullanıcı verilerini işler ve kaydeder.
  - + girisYap(email: String, sifre: String) : boolean -> Doğrulama sonucunu döndürür (true/false).
  - + cikisIstegi() : void -> Çıkış sürecini başlatır

| <b>C</b> KimlikYoneticisi                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• kayitOl(adSoyad : String, email : String, sifre : String, uni : String) : void</li> <li>• girisYap(email : String, sifre : String) : boolean</li> <li>• cikisIstegi() : void</li> </ul> |

#### 3.4.3.2 YorumYoneticisi

- Açıklama: Yorum verilerini kontrol eder.
- Metotlar:
  - + yorumEkle(postId: int, yazarId: int, icerik: String) : void -> Yorumu oluşturur ve ilgili gönderiye ekler.



- + yorumSil(yorumId: int) : boolean -> Silme işleminin başarı durumunu döndürür.

|  YorumYonetici                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>● yorumEkle(postId : int, yazarId : int, icerik : String) : void</li> <li>● yorumSil(yorumId : int) : boolean</li> </ul> |

### 3.4.3.3. FeedYonetici

- Açıklama: Ana akış algoritmasını ve kullanıcı takip süreçlerini yürütür.
- Metotlar:
  - + kullanıcıTakipEt(kullaniciId: int, hedefId: int) : boolean -> Takip etme işlemini gerçekleştirir ve sonucu döndürür.
  - + anaAkisiOlustur(kullaniciId: int) : Feed -> Kullanıcıya özel akış nesnesini oluşturup döndürür.

|  FeedYonetici                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>● kullanıcıTakipEt(kullaniciId : int, hedefId : int) : boolean</li> <li>● anaAkisiOlustur(kullaniciId : int) : Feed</li> </ul> |

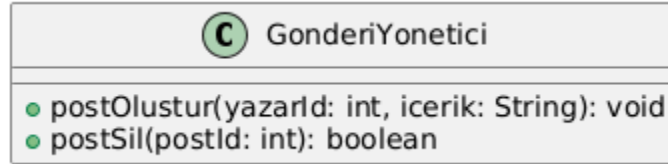
### 3.4.3.4 BeğeniYonetici

- Açıklama: Gönderilere gelen beğenileri yönetir ve veri tutarlılığını sağlar.
- Metotlar:
  - + beğeniEkle(postId: int, kullaniciId: int) : void -> İlgili gönderiye beğeni ekler.
  - + beğeniKaldır(postId: int, kullaniciId: int) : void -> Mevcut beğeniyi sistemden siler.

|  BeğeniYonetici                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>● beğeniEkle(postId: int, kullaniciId: int): void</li> <li>● beğeniKaldır(postId: int, kullaniciId: int): void</li> </ul> |

### 3.4.3.5 GonderiYonetici

- Açıklama: Gönderi (Post) yaşam döngüsünü kontrol eder.
- Metotlar:
  - + postOlustur(yazarId: int, icerik: String) : void -> Yeni gönderiyi işler ve sisteme ekler.
  - + postSil(postId: int) : boolean -> Silme durumunu döndürür.



### 3.4.4. Varlık (Entity) Sınıfları

Bu sınıflar sistemin veri tabanındaki karşılıklarıdır ve sadece bilgi depolamak için kullanılırlar.

#### 3.4.4.1 Kullanici

Sistemdeki öğrencilerin dijital kimlik bilgilerini ve sosyal ağlarını tutar.

- kullaniciId: Kullanıcıyı diğerlerinden ayıran benzersiz kayıt numarasıdır.
- kullaniciAdi: Profilde görünen ve aramalarda kullanılan kullanıcı ismidir.
- email: Giriş yapmak için kullanılan resmi iletişim adresidir.
- sifreHash: Şifrenin güvenlik amacıyla şifrelenmiş (hashlenmiş) halidir.
- takipEdilenler: Kullanıcının takip ettiği kişilerin ID listesidir.



#### 3.4.4.2 Feed (Akış)

Kullanıcıya özel hazırlanan içerik havuzunu temsil eder.

- kullaniciId: Bu akışın hangi öğrenciye ait olduğunu belirtir.
- akisGonderileri: Takip edilen kişilerin gönderilerinden oluşan listedir.

| <b>C</b> Feed                                                                                        |
|------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> kullanıcild: int<br><input type="checkbox"/> akisGonderileri: List<Gonderi> |

#### 3.4.4.3 Gonderi (Post)

Öğrencilerin paylaştığı metinsel içeriklerin ana yapısıdır.

- gonderiId: Paylaşılan gönderinin sistemdeki benzersiz ID'sidir.
- yazarId: Gönderiyi paylaşan kişinin kullanıcı numarasıdır.
- icerik: Paylaşımın içinde yer alan asıl metin mesajıdır.
- tarih: Paylaşımın yapıldığı gün ve saat bilgisidir.

| <b>C</b> Gonderi                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> gonderild: int<br><input type="checkbox"/> yazarId: int<br><input type="checkbox"/> icerik: String<br><input type="checkbox"/> tarih: DateTime |

#### 3.4.4.4 Yorum

Gönderilere yapılan geri bildirimlerin verilerini saklar.

- yorumId: Her yoruma verilen özel kimlik numarasıdır.
- gonderiId: Yorumun hangi gönderiye ait olduğunu belirten ID'dir.
- yazarId: Yorumu yazan öğrencinin ID bilgisidir.
- icerik: Yorum metninin kendisidir.
- tarih: Yorumun sisteme girildiği zamandır.

| <b>C</b> Yorum                                                                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> yorumId: int<br><input type="checkbox"/> gonderild: int<br><input type="checkbox"/> yazarId: int<br><input type="checkbox"/> icerik: String<br><input type="checkbox"/> tarih: DateTime |

#### 3.4.4.5 Beğeni

Kullanıcılar ile gönderiler arasındaki beğeni etkileşimini belgeler.

- beğeniId: Beğeni eyleminin kendi kayıt numarasıdır.
- gonderiId: Beğenilen gönderinin ID'sidir.
- kullanıcıId: Beğen butonuna basan kişinin ID'sidir.
- tarih: Beğenin yapıldığı zaman damgasıdır.

| C Beğeni                                                                                                                                                                   |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> beğeniId: int<br><input type="checkbox"/> gonderiId: int<br><input type="checkbox"/> kullanıcıId: int<br><input type="checkbox"/> tarih: DateTime |

#### 3.4.4.6 Oturum (Session)

Kullanıcının sisteme giriş yaptıktan sonraki aktiflik sürecini yönetir.

- aktifKullanıcıId: Şu an uygulamayı kullanan kişinin ID'sidir; sistem bu sayede "kendi profilini" gösterir.
- baslangicZamani: Giriş yapıldığı andır; oturum süresini kontrol etmek için kullanılır.
- token: Şifre girmeden içeride kalmayı sağlayan güvenli erişim anahtarıdır.
- aktifMi(): Token ve süreye bakarak oturumun hâlâ geçerli olup olmadığını kontrol eder.

| C Oturum                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------|
| <input type="checkbox"/> aktifKullanıcıId: int<br><input type="checkbox"/> baslangicZamani: DateTime<br><input type="checkbox"/> token: String |
| <input checked="" type="checkbox"/> aktifMi(): boolean                                                                                         |

#### 3.4.5. Veri Yönetimi ve Kalıcılık (Persistence)

Sistem, verilerin sürekliliğini sağlamak için dosya tabanlı bir yapı kullanır. Python dilinin sağladığı veri yapıları ile süreç şu adımlarla işler:

- Veri Yükleme: Web sitesi her başlatıldığında veya sayfa yüklendiğinde, JsonIsleyicisi aracılığıyla .json dosyalarındaki veriler Python Dictionary (Sözlük) yapısına aktarılır. Böylece taze veriler belleğe alınır.
- Veri İşleme: Çalışma zamanında (runtime) tüm aramalar, eşleştirmeler ve güncellemeler bu sözlükler üzerindeki benzersiz ID alanları üzerinden yapılır.

- Veri Kaydetme: Control katmanındaki metotlar (örn: kayıtOl, postOlustur) tetiklendiğinde, bellekteki güncel sözlük yapısı tekrar JSON formatına dönüştürülerek dosyaya yazılır.

### 3.4.6. Yardımcı (Utility) Sınıf

#### 3.4.6.1 JsonIsleyicisi

- Açıklama: Python sözlükleri ile fiziksel JSON dosyaları arasındaki çift yönlü veri akışını yöneten köprü sınıftır.
- Metotlar:
  - + veriOku(dosyaYolu: String) : Dictionary -> Dosyadan okunan veriyi parse ederek Python sözlüğüne dönüştürür.
  - + veriYaz(dosyaYolu: String, veri: Dictionary) : boolean -> Sözlükteki güncel durumu JSON dosyasına formatlı olarak kaydeder.

|  JsonIsleyicisi                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>• veriOku(dosyaYolu: String): Dictionary</li> <li>• veriYaz(dosyaYolu: String, veri: Dictionary): boolean</li> </ul> |

### 3.4.7. Tasarım Kararları ve İlişkiler

- Katmanlı Mimari: Boundary (Arayüz) sınıfları hiçbir veriyi doğrudan işlemez, sadece Control (Yönetici) sınıflarına istek gönderir (Bağımlılık/Dependency ilişkisi). İş mantığını Control sınıfları yürütür.
- Utility Bağımlılığı (<>): Tüm Control (Yönetici) sınıfları, veri bütünlüğünü ve dosya tabanlı kayıt sistemini sürdürebilmek için doğrudan JsonIsleyicisi sınıfını kullanır.
- Çokluk (Multiplicity) ve Bağlantı İlişkileri:
  - Kullanıcı - Takip: Bir kullanıcı birden fazla kullanıcıyı takip edebilir (0..\* reflexive / öz yinelemeli ilişki).
  - Kullanıcı - Oturum: Bir kullanıcının aktif olarak sahip olduğu bir (0..1) Oturumu bulunur.
  - Kullanıcı - Gönderi: Bir kullanıcının birden fazla gönderisi olabilir (1 -> 0..\* paylaşır ilişkisi).

- Gönderi - Yorum/Beğeni: Bir gönderinin altında birden fazla yorum ve beğeni barınabilir (1 -> 0..\* alır/içerir ilişkisi).
- Feed - Gönderi: Bir Feed (Akış), içerisinde birden fazla gönderiyi listeler (barındırır).

### 3.5. Tasarım Desenleri

UniSocial platformunun geliştirilme sürecinde; sistemin sürdürülebilirliğini sağlamak, karmaşıklığı yönetmek ve özellikle harici bir veritabanı olmaması gibi kısıtlayıcı işlevsel olmayan gereksinimleri (non-functional requirements) karşılamak amacıyla çeşitli yazılım tasarım desenleri (design patterns) ve mimari prensipler entegre edilmiştir. Projede uygulanan temel tasarım desenleri ve tercih edilme nedenleri aşağıda detaylandırılmıştır:

- **Singleton (Tekil) Deseni:**
  - **Uygulandığı Sınıf:** VeriDeposu / JsonIsleyicisi
  - **Tercih Nedeni ve Uygulanışı:** Projenin işlevsel olmayan sözde gereksinimlerinde belirtildiği üzere, sistemde ilişkisel bir veritabanı kullanılmayacaktır. Veriler, uygulamanın çalışma zamanında (runtime) Python listeleri ve sözlüklerinde (Örn: *TumKullanicilar* listesi, *TumGonderiler* listesi) tutulmaktadır. Uygulamanın yaşam döngüsü boyunca bu veri havuzunun sıfırlanmaması ve uygulamanın tüm katmanlarından aynı veri listelerine senkronize bir şekilde erişilebilmesi kritik bir gereksinimdir. Bu durumu güvenli bir şekilde yönetmek için, bellek üzerinde yalnızca tek bir örneği (instance) oluşturulan merkezi bir yapı tasarlanmış ve Singleton deseni ile korunmuştur. Bu "zeki çözüm" sayesinde, uygulamanın farklı yöneticilerinden (Control sınıfları) gelen eşzamanlı okuma/yazma istekleri aynı bellek adresi üzerinde işlenerek veri tutarsızlığının önüne geçilmiştir.
- **MVC (Model-View-Controller) Adaptasyonu:**
  - **Uygulandığı Sınıflar:** Tüm Varlık (\*Entity), Yönetici (\*Yoneticici) ve Arayüz/HTML sayfaları.
  - **Tercih Nedeni ve Uygulanışı:** Uygulama, Flask framework'ünün doğası gereği MVC prensiplerine yakın bir mimaride kurgulanmıştır.
    - **View (Görünüm):** HTML sayfaları ve kullanıcı etkileşimlerini yakalayan ErisimArayuzu, AnaAkisArayuzu bileşenleri bu katmanı oluşturur.

- **Controller (Kontrolcü):** Sistemin iş mantığını yürüten Yönetici sınıflardır (Örn: KimlikYönetici, GonderiYönetici). View'dan gelen HTTP isteklerini yakalar, Model üzerindeki verileri günceller.
  - **Model (Veri Modeli):** İş nesnelerini temsil eden Varlık sınıflarıdır (Kullanici, Gonderi vb.). Bu yapı sayesinde kodun ayrıştırılabilirliği (separation of concerns) sağlanmış ve projenin modülerliği artırılmıştır.
- **Facade (Görünüş/Cephe) Deseni:**
    - **Uygulandığı Sınıf:** JsonIsleyicisi
    - **Tercih Nedeni ve Uygulanışı:** Sistemde verilerin kalıcılığını sağlamak için JSON dosya sistemi kullanılmaktadır. Dosya okuma/yazma, veri dönüştürme (parsing) ve I/O hata yönetimi gibi alt seviye işlemler karmaşıktır. JsonIsleyicisi sınıfı, bu karmaşıklığı diğer yönetici sınıflardan (Controller) gizleyerek onlara sadece veriOku() ve veriYaz() gibi basit, sadeleştirilmiş bir arayüz sunar. Bu durum, diğer sınıfların dosya sisteminin detaylarını bilmeden veriye güvenle erişmesini sağlayan başarılı bir Facade deseni uygulamasıdır.
  - **Strategy (Strateji) Deseni (Yapısal Adaptasyon):**
    - **Uygulandığı Bölüm:** Veri Saklama ve Erişim Mantığı
    - **Tercih Nedeni ve Uygulanışı:** Projenin mevcut aşamasında veritabanı yerine JSON tabanlı dosya sistemi stratejisi benimsenmiştir. Ancak mimari, **JsonIsleyicisi** üzerinden izole edildiği için, ilerleyen süreçlerde gerçek bir ilişkisel veritabanına (Örn: MySQL, PostgreSQL) geçiş yapılmak istendiğinde sistemin iş mantığını (Controller) ve varlıklarını (Entity) değiştirmeye gerek kalmayacaktır. Yalnızca veri kaydetme "stratejisi" değiştirilerek uygulamanın esnekliği garanti altına alınmıştır.
  - **BCE (Boundary-Control-Entity) Deseni:**
    - **Uygulandığı Bölüm:** Sistemin Tüm Sınıf Hiyerarşisi
    - **Tercih Nedeni ve Uygulanışı:** Sistemdeki sınıfların sorumlulukları; Arayüz (Boundary), İş Mantığı (Control) ve Varlık (Entity) olmak üzere keskin çizgilerle birbirinden ayrılmıştır. Boundary sınıfları doğrudan veri işlemez, Entity sınıfları ise iş mantığı barındırmaz; aradaki köprüyü Control sınıfları kurar. Bu desenin titizlikle uygulanması, projenin yüksek içsel tutarlılığa (high cohesion) ve düşük bağımlılığa (low coupling) sahip olmasını sağlamıştır.

## Sözlük (Glossary)

- **Arayüz (Boundary / UI):** Kullanıcının sistemle etkileşime geçtiği, web tarayıcısı üzerinden görüntülenen görsel HTML/CSS bileşenleri.
- **CRUD (Create, Read, Update, Delete):** Veri üzerinde gerçekleştirilen temel oluşturma, okuma, güncelleme ve silme işlemlerinin kısaltmasıdır.
- **Çerçeve (Framework):** Yazılım geliştirme sürecini standartlaştıran ve hızlandıran önceden hazırlanmış kod altyapısı. Bu projede web altyapısı için Flask kullanılmıştır.
- **Dosya Tabanlı Veri Saklama (File-Based Storage):** Projede harici bir SQL veritabanı kullanılmadığı için, verilerin kalıcılığını sağlamak amacıyla yerel JSON dosyalarında saklanması yaklaşımı.
- **Düzenli İfade (Regex - Regular Expression):** Metin blokları içinde belirli bir deseni aramak ve doğrulamak için kullanılan kurallar dizini. Projede .edu uzantılı e-posta kontrolü için kullanılmıştır.
- **Düşük Bağlılık ve Yüksek İç Uyum (Low Coupling & High Cohesion):** Sistem modüllerinin birbirine minimum seviyede bağımlı olması (Low Coupling) ve her modülün yalnızca kendi sorumluluğuna odaklanması (High Cohesion) yazılım tasarım prensibi.
- **Entity (Varlık):** Sistemdeki temel iş nesnelerini ve veri modellerini temsil eden sınıf yapılarıdır (Örn: Kullanıcı, Gonderi).
- **Facade (Ön Yüz) Deseni:** Karmaşık alt sistemlerin detaylarını (örneğin JSON dosya okuma/yazma işlemlerini) gizleyerek, dışarıdaki diğer sınıflara kullanımı kolay bir arayüz sunan tasarım deseni.
- **İstemci-Sunucu Mimari (Client-Server Architecture):** Kullanıcının cihazındaki tarayıcının (istemci) ağ üzerinden merkezi bir sunucuya istek gönderdiği ve yanıt aldığı mimari yapı.
- **JSON (JavaScript Object Notation):** Veri saklamak ve taşımak için kullanılan, metin tabanlı ve hafif veri yapılandırma formatı.
- **Katmanlı Mimari (Layered Architecture):** Kod karmaşasını önlemek için yazılımın Arayüz, İş Mantığı ve Veri gibi farklı sorumluluklara sahip alt birimlere bölünerek geliştirilmesi.
- **Mockup:** Yazılımın kullanıcı arayüzünün tamamlandığında nasıl görüneceğini tasvir eden statik ön tasarım çizimleri.
- **MVC (Model-View-Controller):** Yazılım sistemlerini veri (Model), görünüm (View) ve iş mantığı/kontrol (Controller) olmak üzere üç ana yapıya ayıran mimari desen.
- **Route / Endpoint:** Web uygulamasında belirli bir işlevi tetiklemek veya bir sayfayı görüntülemek için istemciye açılan URL adresleri.



- **Singleton Deseni:** Bir sınıftan uygulama yaşam döngüsü boyunca yalnızca tek bir nesne oluşturulmasını garanti eden, projede veri tutarsızlığını önlemek için kullanılan tasarım deseni.
- **UML (Unified Modeling Language):** Yazılım sistemlerinin mimarisini, yapısını ve davranışlarını görselleştirmek için kullanılan standart modelleme dili.
- **Zaman Damgası (Timestamp):** Sistemde oluşturulan bir gönderinin veya yorumun, sunucu tarafından kaydedildiği kesin tarih ve saat bilgisini tutan veri formatı.

## Referanslar

[1] E. Tuzun, "Sample Design Report Fall 2021," Bilkent University, Department of Computer Engineering, 2021. [Online]. Available: <https://www.cs.bilkent.edu.tr/~eraytuzun/teaching/cs319/SampleProjects/Fall2021-Sample-DesignReport.pdf>

[2] E. Tuzun, "Sample Design Report Fall 2022," Bilkent University, Department of Computer Engineering, 2022. [Online]. Available: <https://www.cs.bilkent.edu.tr/~eraytuzun/teaching/cs319/SampleProjects/Fall2022-Sample-DesignReport.pdf>

[3] Y. Itepli et al., "LOST Project Design Report," GitHub repository, 2022. [Online]. Available: [https://github.com/yitepli/CS319Project\\_Group2D\\_LOST/blob/master/Reports/Submissions/DesignReport/Design\\_Report\\_Final.pdf](https://github.com/yitepli/CS319Project_Group2D_LOST/blob/master/Reports/Submissions/DesignReport/Design_Report_Final.pdf)